WNDRVR
SUPPORT NETWORK

VXWORKS 7 WEB SERVICES PROGRAMMER'S GUIDE, SR0650

Copyright Notice

Copyright © 2022 Wind River Systems, Inc.

All rights reserved. No part of this publication may be reproduced or transmitted in any form or by any means without the prior written permission of Wind River Systems, Inc.

Wind River, Simics, Tornado, and VxWorks are registered trademarks of Wind River Systems, Inc. Helix, Pulsar, Rocket, Titanium Cloud, Titanium Control, Titanium Core, Titanium Edge, Titanium Edge SX, Titanium Server, and the Wind River logo are trademarks of Wind River Systems, Inc. Any third-party trademarks referenced are the property of their respective owners. For further information regarding Wind River trademarks, please see:

www.windriver.com/company/terms/trademark.html

This product may include software licensed to Wind River by third parties. Relevant notices (if any) are provided for your product on the Wind River download and installation portals:

https://delivers.windriver.com/

https://windshare.usa.windriver.com/

Wind River may refer to third-party documentation by listing publications or providing links to third-party websites for informational purposes. Wind River accepts no responsibility for the information provided in such third-party documentation.

Corporate Headquarters

Wind River
500 Wind River Way
Alameda, CA 94501-1153
U.S.A.
Toll free (U.S.A.): +1-800-545-WIND
Telephone: +1-510-748-4100
Facsimile: +1-510-749-2010

For additional contact information, see the Wind River website:

www.windriver.com

For information on how to contact Customer Support, see:

www.windriver.com/support

VxWorks 7 Web Services Programmer's Guide, SR0650

10 June 2020

1. WEB SERVICES OVERVIEW

1. Introduction

The VxWorks® 7 Web Services Programmer's Guide describes the use of VxWorks Web Services, which provides development and run-time facilities for creating client and server SOAP applications, as well as XML applications.

Simple Object Access Protocol (SOAP) is a lightweight protocol based on Extensible Markup Language (XML) and Hypertext Transfer Protocol (HTTP) that is used to exchange information in a distributed environment and to invoke methods on servers and other objects.

Your applications can use Web Services to exchange SOAP messages with any system that supports Web Services, such as systems running Microsoft .NET or Apache Axis. This requires a knowledge of the C programming language, XML and SOAP technologies, Wind River Workbench development facilities, and your target architecture.

2. XML and SOAP

Web Services implements Extensible Markup Language (XML) and Simple Object Access Protocol (SOAP) technologies.

Extensible Markup Language (XML)

A markup language is a mechanism to identify structures inside a set of data. XML is a metadata language that defines a syntax for which field-specific markup languages can be written. It provides a set of rules for defining semantic tags that organize a document into structured information.

XML is similar to Hypertext Markup Language (HTML) in that both are markup languages; however, XML has been specifically designed to deliver richly structured data over the Internet. HTML defines a fixed set of tags that described a fixed number of elements. Unlike HTML, XML does not define a specific set of tags but rather sets a syntax through which you can define field-specific tags as needed. While HTML is most commonly used for presentational markup (to control the presentation of information), XML is primarily for data markup (to organize a document into structured information).

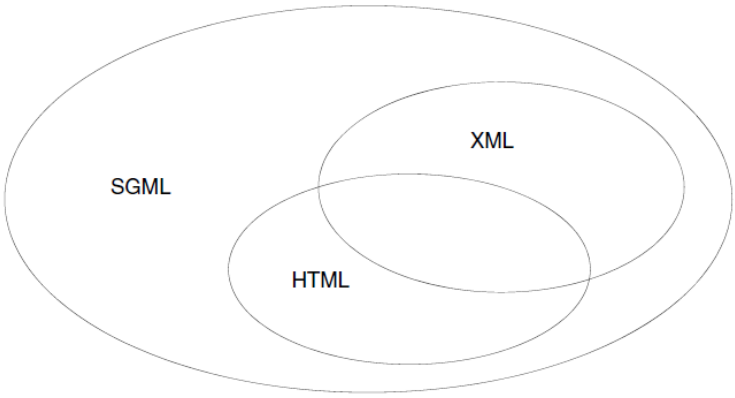
XML use provides many benefits:

- XML documents are human readable. The language is internally consistent, so you can read an XML document to understand its content.
- There are a number of XML support technologies that provide for the interpretation and transformation of XML data for web page display and report generation.
- XML is non-proprietary and easy to read and write. This facilitates interpretability and the exchange of information.

Standardized Generalized Markup Language (SGML) is used to define markup languages for documents. From it was derived HTML and XML. XML is a subset of SGML and includes aspects of HTML.

Figure 1. Relationship of XML to SGML and HTML

WNDRVR



Simple Object Access Protocol (SOAP)

SOAP is an XML-based protocol for exchanging messages between applications in a decentralized, distributed environment. The SOAP message format consists of an envelope that defines a framework for describing what is in a message and how to process it, a set of encoding rules for expressing instances of application-defined data types, and a convention for representing remote procedure calls and responses. All SOAP encoding is in XML. SOAP messages are typically transferred over HTTP over Transmission Control Protocol (TCP).

SOAP is essentially a high-level method for machines to tell other machines to do things over the Internet. It increases interoperability by enabling scripted web applications to cross operating system boundaries; for example, connecting desktop tools to powerful content management and collaborative systems running on servers.

3. Use Cases

Web Services is a family of standards-based facilities that provide XML run-time facilities, SOAP run-time facilities, and SOAP application code generators.

Web Services applications may be applied in the following use cases.

Industrial Design

- Sharing data between real time embedded controllers and a desktop in an operation center
- Distributing data between pieces of test equipment on a lab bench
- Initiating actions and procedures between remote systems
- Storing application data in a configuration file
- Communicating between industrial controllers on a factory floor and the applications running on a desktop in a .NET environment in an operations center

Consumer Devices

- Configuring a print job and bill for the task from the same application
- Collecting data on devices and transmitting that data to servers
- Driving devices from remote user applications
- Enabling Web Services for consumer devices such as multi-function printers (MFPs), set-top boxes (STB), and VoIP terminals.

Network Equipment

- Managing and configuring a network device, such as a router, access point, or set-top box
- Reporting statistics to a monitoring system
- Enabling Web Services-based management of network equipment

4. Additional Resources

Before you begin using Web Services, you must be familiar with the C programming language and with XML and SOAP technologies. You must also be familiar with Wind River Workbench development facilities (GUI or CLI) and your target architecture.

View the following resources for additional information.

Online Resources

You can refer to the W3C Recommendations or IETF Request for Comments (RFC) for any protocol or concept on which you are working.

- The World Wide Web Consortium, http://www.w3c.org
- W3 Schools, http://www.w3schools.com/
- *The Barebones Guide to HTML*, http://werbach.com/barebones/
- The Internet Engineering Task Force, http://www.ietf.org
- The Web Services Interoperability Organization, http://www.ws-i.org

Books

View the following references for information relevant to your requirements:

- *Internetworking with TCP/IP, Volume I: Principles, Protocols, and Architecture*, Douglas E. Comer.
- *The Simple Book: An Introduction to Management of TCP/IP-based Internets*, Marshall T. Rose.
- *SNMP Network Management*, Paul Simoneau.
- *A Practical Guide to SNMPv3 and Network Management*, David Zeltzerman.
- *SNMP, SNMPv2, SNMPv3, and RMON 1 and 2*, William Stallings.
- *Understanding SNMP MIBs*, David Perkins.

XML Specifications

Web Services supports the following XML-related standards:

- ML 1.0, as supported by Expat 2.0.1, defined in *Extensible Markup Language (XML) 1.0, Second Edition* (http://www.w3.org/TR/REC-xml)
- Canonical XML 1.0, as supported by Expat 2.0.1, defined in *Canonical XML Version 1.0* (http://www.w3.org/TR/xml-c14n)
- *Namespaces in XML* (http://www.w3.org/TR/REC-xml-names)

SOAP Specifications

Web Services supports the following SOAP-related standards:

WNDRVR

- SOAP 1.2 as supported in *gSOAP 2.8.14: Generator Tools for Coding SOAP/XML Web Service and Client Applications in C and C++*, defined by the following documents:
  - *SOAP Version 1.2 Part 0: Primer* (http://www.w3.org/TR/soap12-part0/)
  - *SOAP Version 1.2 Part 1: Messaging Framework* (http://www.w3.org/TR/soap12-part1/)
  - *SOAP Version 1.2 Part 2: Adjuncts* (http://www.w3.org/TR/soap12-part2/)
  - *SOAP Version 1.2 Specification Assertions and Test Collection* (http://www.w3.org/TR/soap12-testcollection)
- *Simple Object Access Protocol (SOAP) 1.2* (http://www.w3.org/tr/soap/)
- Web Services Interoperability Organization (WS-I) Basic Profile 1.0 (http://www.ws-i.org/Profiles/BasicProfile-1.0.html)
- Organization for the Advancement of Structured Information Standards (OASIS) Web services security standards (WS-Security) 1.0:
  - SOAP Message Security 1.0
  - Username Token Profile 1.0
  - X.509 Token Profile 1.0
  - XML **secext.xsd** and **utility.xsd** schema files associated with the above security standards

RFCs and IEEE Specifications

The following IETF RFCs are relevant to Web Services and can be found at the IETF Web site, http://www.ietf.org.

Web-Related RFCs include:

- RFC 1867: *Form-based File Upload in HTML*
- RFC 2616: *Hypertext Transfer Protocol -- HTTP/1.1*
- RFC 2617: *Basic and Digest Access Authentication*
- RFC 2376: *XML Media Types*

SNMP-Related RFCs include:

- RFC 1155: *Structure and Identification of Management Information for TCP/IP-based Internets*
- RFC 1157: *Simple Network Management Protocol (SNMP)*
- RFC 1212: *Concise MIB Definitions*
- RFC 1213: *Management Information Base for Network Management of TCP/IP-based internets: MIB-II*
- RFC 2578: *Structure of Management Information Version 2 (SMIv2)*
- RFC 2580: *Conformance Statements for SMIv2*
- RFC 3411: *An Architecture for Describing Simple Network Management Protocol (SNMP) Management Frameworks*
- RFC 3412: *Message Processing and Dispatching for the Simple Network Management Protocol (SNMP)*
- RFC 3413: *Simple Network Management Protocol (SNMP) Applications*
- RFC 3414: *User-based Security Model (USM) for version 3 of the Simple Network Management Protocol (SNMPv3)*
- RFC 3415: *View-based Access Control Model (VACM) for the Simple Network Management Protocol (SNMP)*

Wind River Documentation

The following Wind River documents present information associated with Web Services.

- VxWorks 7 Configuration and Build Guide—describes how to install and build components of the Wind River VxWorks Platforms product
- VxWorks 7 CLI, Web, MIBway Programmer's Guide—provides online reference descriptions of all CLI, Web, and MIBway libraries

Each library entry lists the routines found in that library, including a one-line synopsis of each, along with a general description of their use. The document also provides complete reference entries for all SNMP user-exits, macros, and routines.

- VxWorks 7 SNMP Programmer's Guide—describes how to use the Wind River SNMP source code. The document also provides a technical description of the product, standards compliancy, and details of the server and client model.

2. WEB SERVICES TECHNOLOGY

1. Introduction

Web Services provides standards-based XML and SOAP technologies to create XML and SOAP applications.

The key elements of Web Services are the following:

- XML run-time facilities
- SOAP run-time facilities
- SOAP application code generators

2. VxWorks XML Facilities

Web Services XML run-time facilities provide enhanced and extended XML application support.

The facilities include the following:

- XML parser
- XML canonicalizer
- XML pull parser
- XML output facility

The XML parser is based on the open source Expat XML parser (for more information, see Wind River Enhancements to the Expat XML Parser on page 8). The other components are Wind River technologies.

The following VxWorks facilities provide kernel support for applications that work with XML:

XML parser

The XML parser is a SAX-like (Simple API for XML) stream-oriented parser in which an application registers callbacks for XML-specific syntax events; for example, start tags, end tags, and **ELEMENT** declarations. The parser provides you the flexibility to configure such things as namespace handling, document encoding, parameter entity parsing, and external entity parsing. The parser also provides error handling, as well as the ability to pass user data throughout calls to your callbacks.

For detailed information regarding the XML parser, see the Expat XML Parser documents provided with your Workbench installation at the following location:

docs/extensions/eclipse/plugins/com.windriver.ide.doc.wr_web_services/ wr_web_services_api_reference_1.8/ xmldoc.html

Alternatively, you can find the documents on the Wind River online support Web site, https://docs.windriver.com/.

XML canonicalizer

The XML canonicalizer module provides the means to re-write an existing (valid) XML document in canonical form, according to the *W3C Canonical XML Version 1.1* specification (for more information, see http://www.w3.org/TR/xml-c14n11). The XML canonicalizer creates all output through the XML output module.

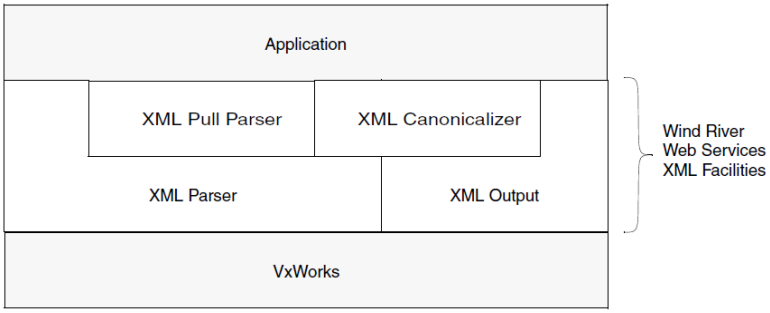
XML output

This XML output module provides a SAX-like API for outputting XML to a file. For example, the routine for handling an XML start element in SAX is XML_StartElementHandler( ), and the routine to write an XML start element in the output module is XML_StartElementWrite( ).

XML pull parser

The XML pull parser API provides routines for making stream-based XML parsing easier by allowing the developer to iterate through an XML document rather than respond to events as is commonly the case with SAX-like parsers.

Figure 2. Web Services XML facilities in VxWorks



SOAP is essentially a high-level method for machines to tell other machines to do things over the Internet. It increases interoperability by enabling scripted Web applications to cross operating system boundaries, for example connecting a device with an enterprise application.

A SOAP system operates as a stateless, one-way message exchange paradigm. If you define the interaction of SOAP messages, you can develop a request and response system using the basic paradigm. The SOAP protocol defines neither the type of data that you can transmit nor the media through which that data is to be transmitted. The SOAP protocol defines the following:

- The structure of SOAP messages
- The required actions that a receiver of a SOAP message must perform
- The methods a SOAP node must use for processing SOAP messages
- The methods by which SOAP intermediaries may change, remove, or modify parts of SOAP messages
- Basic SOAP message parts, such as envelope, header, and body
- How to send SOAP messages via HTTP POST and HTTP GET operations
- How to send SOAP messages within SMTP-standard e-mail messages

SOAP also describes how to send structured messages with a decentralized system of SOAP nodes. SOAP defines an extensible message framework, containing a message construct, which can be transmitted over a variety of underlying network protocols. SOAP is intended to be both simple and extensible. SOAP does not define aspects of messaging related to security, routing, or reliability of transmission.

For more information on the SOAP protocol, see http://www.w3.org/tr/soap.

Security Standards

Web Services conforms to the following OASIS Web Services Security (WS-Security) standards:

- SOAP Message Security 1.0
- Username Token Profile v1.0
- X.509 Token Profile

- The XML schema files associated with these standards

The WS-Security specifications enhance existing SOAP messaging to allow you to implement security functions for your Web Services message exchanges. They outline a standardized security framework for Web Services that use SOAP messages, while accommodating various security models and encryption methods. The standards cover such security issues as authentication, authorization, and confidentiality, and provide a mechanism for associating a security token with message content. The specifications are designed to be extensible, to support several (or even several concurrent) security token formats.

For more information on the WS-Security standards, see http://www.oasis-open.org/specs/index.php#wssv1.1.

3. Wind River Enhancements to the Expat XML Parser

Wind River has enhanced the open source Expat 2.0.1 XML parser.

It has provided the following enhancements:

- Fixes for DTD parsing bugs in the Expat code
- Clarification of some code comments for readability
- Additional performance and robustness testing
- Additional UTF-8 encoding testing
- An XML pull parser API that complements the SAX API
- An XML output API that corresponds to the SAX parsing handlers
- Integration of XML functions with VxWorks
- Development tools integrated with Wind River Workbench
- Demonstrative XML tutorials and examples
- Online API reference documentation
- Design and programming information in this document

Note: This release of Web Services did not produce any violations when tested with the XML 1.0 Conformance test suite (which includes over 2000 tests).

4. SOAP Run-Time Facilities

The Web Services SOAP run-time facilities provide a native gSOAP Web server and XML parser, which provide advantages over the standalone components in terms of size and speed.

The SOAP run-time facilities do the following:

- Handle HTTP requests and responses
- Parse and produce XML (specifically SOAP messages)
- Translate between C/C++ and XML data structures

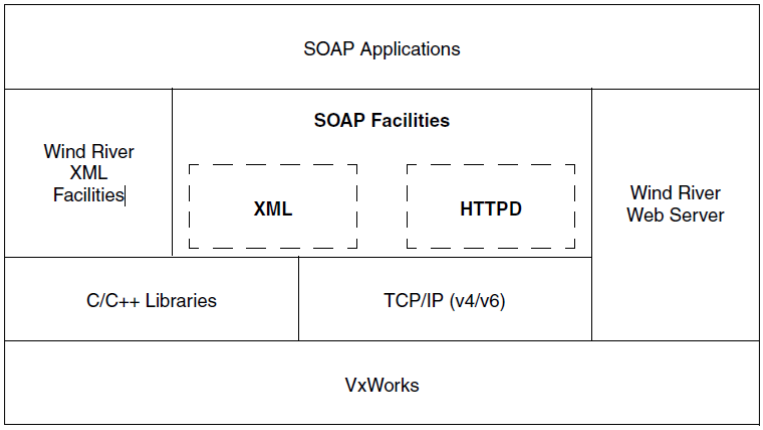
The SOAP run-time facilities are provided in source code and must be built as part of a VxWorks application (either a kernel application or an RTP application).

For additional functionality, you can use the zlib library and Web Services XML facilities with your applications to provide additional capabilities. For more information, see Using the zlib Library on page 35.

Executing SOAP applications in either the kernel or in a real-time process (RTP) often requires configuring the kernel with additional components (for example, for RTP support, non-default networking configurations, and so on). For more information, see VxWorks 7 Layers and Package Management.

The following figure illustrates the relationship between SOAP applications, the run-time components included in Web Services, and related Wind River facilities.

Figure 3. SOAP run-time environment



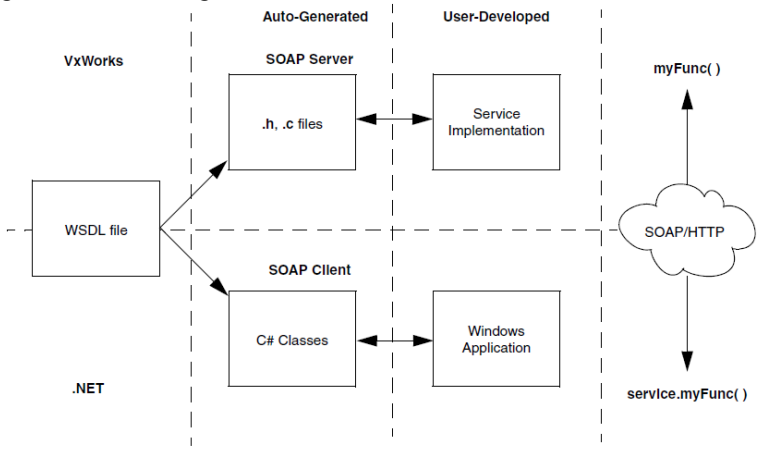
5. VxWorks SOAP Facilities

The basic SOAP run-time facilities include a native XML parser and HTTP server, which are generated specifically for an application.

They have limited capabilities, but are also smaller and faster than the optional components. The SOAP libraries are based on the open source gSOAP toolkit. For more information, see Wind River Enhancements to gSOAP on page 10.

The following figure shows the typical sequence for creating a Web Services client and server, starting with a Web Services Description Language (WSDL) file and ending with the implementation of server and client functions (myFunc( ) and service.myFunc( )). A WSDL file describes how to access a Web service and what operations it will perform. It provides for interoperability with other enterprise Web services applications by describing those services in a platform-independent, machine-readable manner. The figure illustrates the parallels between Web Services and other enterprise Web services technologies.

Figure 4. SOAP design environment



6. Wind River Enhancements to gSOAP

Wind River has enhanced the gSOAP 2.8.14 software development toolkit.

It provides the following enhancements:

- Integration of SOAP operations with VxWorks
- A graphical user interface for the WSDL importer and the SOAP source code generator, integrated with Workbench
- Additional SOAP tutorials and examples
- Integration with the Wind River Web Server
- Online API reference documentation
- Design and programming information in this document

3. CONFIGURING VXWORKS FOR XML AND SOAP

1. Introduction

Web Services lets you configure XML facilities to optimize compilation and requires you to configure VxWorks to support XML and SOAP facilities.

2. Configuring VxWorks XML Facilities

You can configure various aspects of the Web Services XML facilities by modifying the preprocessor macro assignments that specify compilation flags.

About This Task

The macros that most often apply to a XML parser application are:

XML_CONTEXT_BYTES

Specify how much context to retain around the current parse point. The default value is 1024.

XML_DTD

Define to make parameter entity parsing functionality available.

XML_MIN_SIZE

Specify this option to make the XML Expat parser image size more compact.

XML_NS

Define to make XML Namespaces functionality available.

The additional macros in the **xmlConfig.h** file are associated with Expat support for operating systems other than VxWorks. Web Services has already optimized these macros for use with VxWorks, so in most cases the defaults should be appropriate.

Procedure

Modify the preprocessor macro assignments in the following file:

installDir **/vxworks-7/pkgs_v2/app/expat/src/xmlConfig.h**

3. Configuring VxWorks with XML Support

To configure VxWorks with support for XML facilities, you must include the required VSB layers and VIP components.

About This Task

VxWorks XML facilities are provided in source code and must be built before they can be used with either kernel or real-time process (RTP) applications.

WNDRVR

Before You Begin

You must know how to use Workbench or the vxprj utility to create and configure VSB and VIP projects. For more information, see VxWorks 7 Configuration and Build Guide.

Procedure

1. Create a VSB project based on the board support package or CPU type needed to support your hardware target.
2. Enable the **EXPAT** layer, along with any layers required for your system.
3. Build the VSB.
4. Create a VIP based on the VSB project.
5. Add the **INCLUDE_XML**_*FACILITY* components to the VIP.
6. Build the VIP.

4. Configuring VxWorks with SOAP Support

To configure VxWorks with support for SOAP facilities, you must include the required VSB layers and VIP components.

About This Task

VxWorks SOAP facilities are provided in source code and must be built before they can be used with either kernel or real-time process (RTP) applications.

In addition to configuring support for SOAP facilities, you must configure VxWorks with support for your applications. For example, you may include components to support the following features:

- Automatically starting applications at run time
- Including a DNS resolver that is configured for your run-time environment to facilitate client-server communication
- Including SSL for secure transactions (SOAP over SSL, or SOAP with WS-Security)
- Including target tools such as the kernel module loader and kernel shell

For information about VxWorks run-time features and their configuration, see *VxWorks 7 Application Programmer's Guide: VxWorks Configuration for Shared Libraries and Plug-ins*.

Before You Begin

You must know how to use Workbench or the vxprj utility to create and configure VSB and VIP projects. For more information, see VxWorks 7 Configuration and Build Guide.

Procedure

1. Create a VSB project based on the board support package or CPU type needed to support your hardware target.
2. Enable the **GSOAP** layer, along with any layers required for your system.
3. Enable the SOAP option that supports your application files:
   - Enable the **FEATURE_GSOAP_C** option for a project in C.
   - Enable the **FEATURE_GSOAP_CPP** option for a project in C++.
4. Build the VSB.

5. Create a VIP based on the VSB project.
6. If you want to use a gSOAP demo server for testing, add the **INCLUDE_GSOAP_SOAPDEMO** component to the VIP.
7. If you want to run your application in the kernel, create and add C source files to your VIP.

   For more information, see SOAP Application Development Tools on page 16.

8. Build the VIP.

4. DEVELOPING XML APPLICATIONS

1. Introduction

Web Services provides kernel support for applications that work with XML.

XML applications can be created either as kernel applications or as real-time process (RTP) applications.

2. Developing XML Applications

After you have configured VxWorks to support XML facilities, you can create kernel or RTP applications that work with XML.

Before You Begin

You must know how to use Workbench or the vxprj utility to create applications. For more information, see *VxWorks 7 Application Programmer's Guide: RTP Applications*.

Procedure

1. If you want to run your application as a DKM or RTP application, create a DKM or RTP project based on the VIP that you created in Configuring VxWorks with XML Support on page 11.
2. If you want to run your application as a DKM or RTP application, add your application code to the DKM or RTP project; if you want to run your application in the kernel without a DKM, add your application code to the VIP.

   For more information on XML application code, see Example XML Application Development on page 14.

3. Add a **webservices/xml/expat.h** include directive to your application source file.

```
#include "webservices/xml/expat.h"
```

4. Build the project.

3. Example XML Application Development

Web Services provides example code from which you can learn basic supported XML functions and the APIs that are most commonly used to parse an XML document. The examples provide a starting point on using the XML parser and the XML pull parser.

This section describes a high-level procedure for developing a typical XML parser application, but additional XML examples are located in the following directory:

installDir **/vxworks-7/pkgs_v2/app/expat-**x.x.x.x **/src/tutorials/examples**

To develop an XML parser application, perform the following steps:

1. Declare the XML parser pointer.
2. Create the XML parser object.
3. Register your custom XML callback functions using the **XML_Set***handler_type* **Handler()** functions.

WNDRVR

4. Develop your custom callback functionality.
   This is where you define the behavior you would like to occur for the events that you have registered callbacks for.
5. Configure the XML parser behavior.
   For more information, see the example tutorials described above.
6. Open the XML input stream (for example, the file or socket to parse) and begin parsing it.
7. Clean up any resources that you used.

WNDRVR

5. DEVELOPING SOAP APPLICATIONS

1. Introduction

Web Services provides source code generation facilities that support SOAP application development.

SOAP facilities generate the basic elements of application code, including header, WSDL, and source files, to develop SOAP client and server applications.

Web Services is based on gSOAP. For detailed information about gSOAP and its use, see *gSOAP User Guide*.

2. SOAP Application Development Tools

Web Services provides tools that automate the production of SOAP header files, application stubs, and skeleton code to facilitate SOAP application development.

For information on the generated files, see SOAP Client Files on page 32 and SOAP Server Files on page 30.

The tools are provided both as independent command-line tools and as integrated GUI facilities in Workbench:

- The WSDL importer, *wsdl2h*, allows you to generate a SOAP header file from a published WSDL document. This header file is then used by the application source code generator.
- The application source code generator, *soapcpp2*, creates C/C++ stub and skeleton code based on a SOAP header file. It can use either a header file generated by the WSDL importer or a manually annotated header file. The source code generator can also create a WSDL document based on the header file that it uses to create the source code.

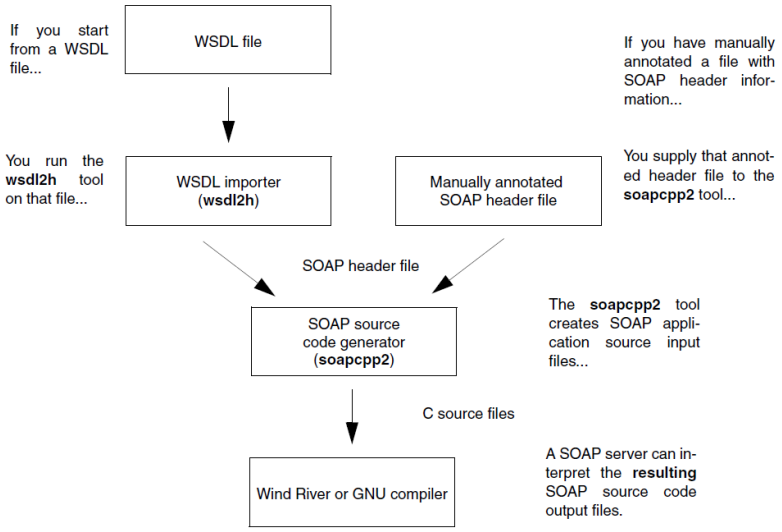
For more information on annotating C header files to support SOAP, see *gSOAP User Guide*.

You can use the SOAP client code that you produce with the source code generator in conjunction with custom application code that makes calls to the generated code and that handles callbacks. As long as you do not modify the generated code, you can regenerate it without risk to the application (for example, if you modify the SOAP interface). Similarly, the generated SOAP server code can be used with custom code that implements SOAP functions.

The following figure illustrates the general method for using the WSDL importer and SOAP source code generator tools to generate executable files to run under VxWorks.

Figure 5. Web Services Tools Workflow

WNDRVR



Workbench Tools on page 17 Workbench provides a graphical interface to the wsdl2h (WSDL importer) and soapcpp2 (SOAP source code generator) utility commands.
SOAP Command-Line Tools on page 21 The wsdl2h WSDL importer and the soapcpp2 SOAP source code generator can be used from the command line to generate a SOAP header file and SOAP source code respectively.
Message Encodings on page 25 Web Services conforms to the interoperability requirements of the Web Services Interoperability Organization (WS-I) Basic Profile 1.0.
SOAP Project Compilation Flags on page 25 Compile SOAP application source files (generated SOAP files and client and server applications) with flags.

2.1. Workbench Tools

Workbench provides a graphical interface to the wsdl2h (WSDL importer) and soapcpp2 (SOAP source code generator) utility commands.

For information on command-line use of the wsdl2h and soapcpp2 utilities, see SOAP Command-Line Tools on page 21.

Generating SOAP Header Files from a WSDL Document on page 17 You can generate a SOAP header file from a WSDL document published by the SOAP service that your client will connect to.
Importing Existing SOAP Header Files on page 19 You can import a file that has been manually annotated with SOAP header information into your project.
Generating SOAP Source Files on page 19 You can generate a C/C++ stub and skeleton code for your SOAP application based on a SOAP header file.

Parent topic: SOAP Application Development Tools on page 16

Generating SOAP Header Files from a WSDL Document

You can generate a SOAP header file from a WSDL document published by the SOAP service that your client will connect to.

About This Task

SOAP client applications require a SOAP header file, which can be generated from a WSDL document or created manually. If you want to import a manually annotated file with SOAP header information, see Importing Existing SOAP Header Files on page 19.

If you want to generate a SOAP header file using the command line, see SOAP Command-Line Tools on page 21.

Before You Begin

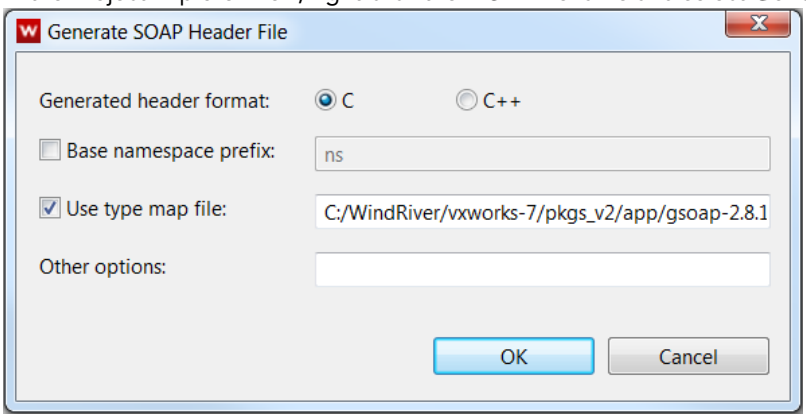
You must have the WSDL document published by a SOAP service.

For information on integrating Web Services with other enterprise Web services technologies using the WSDL importer facilities, see Integrating with Other Web Services on page 34.

You must have configured VxWorks with SOAP support and created a VIP, RTP, or DKM project for your application. For more information, see Configuring VxWorks with SOAP Support on page 12.

Procedure

1. In the Project Explorer view, right-click the project that contains your application and select **Import**.
2. Expand **General**, select **File System**, and click **Next**.
3. Browse to the WSDL file directory and click **OK**.
4. Select the WSDL file and click **Finish**.
5. In the Project Explorer view, right-click the WSDL filename and select **Generate SOAP header**.



6. Configure the relevant header file properties for your application.

Table 1. Fields in the Generate SOAP Header File Dialog Box

Dialog Field	Description
Generated header format	Generates either C or C++ headers. This corresponds to the -c command-line option.
Base namespace prefix	Uses the string entered in this field as the base namespace prefix instead of ns. This corresponds to the -n command-line option.
Use type map file	Uses the named type map file instead of the default **typemap.dat**. This corresponds to the -t command-line option.
Other options	Additional command-line options can be entered here.

For information on command-line options, see SOAP Command-Line Tools on page 21.

The generated file appears in the project directory and in the Project Explorer view.

Parent topic: Workbench Tools on page 17

Importing Existing SOAP Header Files

You can import a file that has been manually annotated with SOAP header information into your project.

About This Task

SOAP client applications require a SOAP header file, which can be generated from a WSDL file or created manually. If you want to generate a SOAP header file from a WSDL document, see Generating SOAP Header Files from a WSDL Document on page 17.

Before You Begin

You must have a manually annotated file with SOAP header information.

You must have configured VxWorks with SOAP support and created a VIP, RTP, or DKM project for your application. For more information, see Configuring VxWorks with SOAP Support on page 12.

Procedure

1. In the Project Explorer view, right-click the project that contains your application and select **Import**.
2. Expand **General**, select **File System**, and then click **Next**.
3. Browse to the SOAP header file directory and click **OK**.
4. Select the SOAP header file and click **Finish**.

Parent topic: Workbench Tools on page 17

Generating SOAP Source Files

You can generate a C/C++ stub and skeleton code for your SOAP application based on a SOAP header file.

Before You Begin

Your application project must contain a SOAP header file. For information on generating a SOAP header file from a WSDL document published by the SOAP service that your client will connect to, see Generating SOAP Header Files from a WSDL Document on page 17. For information on importing an existing SOAP header file, see Importing Existing SOAP Header Files on page 19.

Procedure

1. In the Project Explorer view, right-click the application header file and select **Generate SOAP source code**.

   The Generate SOAP sources dialog box appears.

2. Use this dialog box to generate SOAP client or server code from a SOAP header file.

WNDRVR

Note: Select the **Generate WSDL and schema files** check box if you want generate a WSDL document for your SOAP server application.

The generated files appear in the project directory and in the Project Explorer view.

Table 1. Fields in the Generate SOAP Sources Dialog Box

Dialog Field	Description
Namespaces and encodings	Generates either SOAP 1.1-compliant or SOAP 1.2-compliant source code. This corresponds to the -1 and -2 command-line options.
Generated source format	Generates either C or C++ source code. This corresponds to the -c command-line option. C++ proxy classes are generated according to the header file that they are generated from.
Add SOAP source files to project	Select the **Server** checkbox to generate **soapServer.c(pp)**, **soapC.c(pp)**, **soapH.h**, and **soapStub.h** in the project. Select the **Client** checkbox to generate **soapClient.c(pp)**, **soapC.c(pp)**, **soapH.h**, and **soapStub.h** in the project. Both boxes may be selected, in which case both server and client code are created. This field corresponds approximately to the -C and -S command-line options. Note that if you specify both -C and -S at the command line, neither client nor server code is generated.
Generate WSDL and schema files	Generates a WSDL file and schema files that describe the SOAP service specified in the header file. This corresponds to the -w command-line option.
Use path(s) for #import	Corresponds to the -I import option. Multiple import directories can be listed, separated by semicolons (;).
Prefix for generated file names	Uses the string entered in the text box as the prefix for all generated files. The default prefix is **soap**. For example, if you select this option and enter **myApp** in the text box, the file name **myAppC.c** is used instead of **soapC.c**. This field corresponds to the -n command-line option.
Other options	Additional command-line options can be entered here.

For information on command-line options, see SOAP Command-Line Tools on page 21.

Parent topic: Workbench Tools on page 17

2.2. SOAP Command-Line Tools

The wsdl2h WSDL importer and the soapcpp2 SOAP source code generator can be used from the command line to generate a SOAP header file and SOAP source code respectively.

Both utilities can be used from the VxWorks development shell without having to set any environment variables.

For additional information on wsdl2h and soapcpp2, see *gSOAP User Guide*.

If you want to generate a SOAP header file in Workbench, see Generating SOAP Header Files from a WSDL Document on page 17.

WSDL Importer: wsdl2h

The wsdl2h utility can be used at the command line with gSOAP options to generate a SOAP header file for a SOAP client application. The utility requires a WSDL document published by the SOAP service that your client will connect to.

For example, the following command produces the **pop3.h** header file, generated from an example WSDL document, in the project directory:

`wsdl2h -c` *installDir* **`\workspace\soapClientRTP`** *installDir* **`\WindRiver\vxworks-7\pkgs_v2\app\gsoap-2.8.15.3\core\src\tutorials\soapWithXML\pop3.wsdl`**

> Note: The Wind River Network Stack includes code to create and maintain an SMTP server but does not include a POP3 server.

The wsdl2h command accepts the following command-line options.

Table 1. wsdl2h Command-Line Options

Option	Description
-a	Generate indexed structure names for local elements with anonymous types.
-b	Add bi-directional operations (duplex ops) to serve one-way responses.
-c	Generate C source code (rather than C++ source code).
-d	Use DOM to populate **xs:any**, **xsd:anyType**, and **xs:anyAttribute** elements.
-e	Do not qualify **enum** names. This option does not produce code that conforms to WS-I Basic Profile 1.0a.
-f	Generate flat C++ class hierarchy.
-g	Generate global top-level element declarations.
-h	Display help information.
-I *path*	Use *path* to find files.

Option	Description
-i	Do not import (advanced option)
-j	Do not generate **SOAP_ENV__Header** and **SOAP_ENV__Detail** definitions.
-k	Do not generate **SOAP_ENV__Header mustUnderstand** qualifiers.
-l	Include license information in output.
-m	Use the **xsd.h** module to import primitive types.
-n *name*	Use *name* as the base namespace prefix instead of ns.
-N *name*	Use *name* as the base namespace prefix for service namespaces.
-o *file*	Output to *file* .
-p	Create polymorphic types with C++ inheritance hierarchy, with base **xsd__anyType**.
-P	Do not create polymorphic types with C++ inheritance hierarchy, with base **xsd__anyType**.
-q *name*	Use *name* for the C++ namespace of all declarations.
-r *host:port*	Connect through proxy *host* and *port* .
-s	Do not generate STL code (no **std::string** and no **std::vector**).
-t *file*	Use the specified type map *file* instead of the default file **typemap.dat**.
-u	Do not generate unions.
-v	Generate verbose output.
-w	Always wrap response parameters in a response structure (<=1.1.4 behavior). By default, the **wsdl2h** utility generates method routine prototypes that do not wrap response parameters in a response structure. Use the -w option to ensure backward compatibility when rebuilding existing SOAP applications that use WSDL. Note that this is a change in the default behavior of this option.
-W	Suppress warnings.

Option	Description
-x	Do not generate **_XML any/anyAttribute** extensibility elements.
-y	Generate **typedef** synonyms for **structs** and **enums**.
-z1	Compatibility with 2.7.6e: generate pointer-based arrays.
-z2	Compatibility with 2.7.15: qualify element/attribute referenced members.
-z3	Compatibility with 2.7.16 to 2.8.7: qualify element/attribute references.
-z4	Compatibility up to 2.8.11: don't generate union structs in **std::vector**.
-_	Do not generate **_USCORE** (replace with **UNICODE _x005f**).

SOAP Source Code Generator: soapcpp2

The soapcpp2 source code generator can be used at the command line with gSOAP options to generate a C/C++ stub and skeleton code and a WSDL file for your SOAP application based on a SOAP header file.

For example, the following command produces source code files in the **soapServerDKM** project from the example **echoString.h** header file in the same project.

```
soapcpp2 -1 -c -w -d installDir\workspace\soapServerDKM\echoString.h
```

The soapcpp2 command accepts the following command-line options.

Table 2. soapcpp2 Command-Line Options

Option	Description
-1	Generate SOAP 1.1 namespaces and encodings.
-2	Generate SOAP 1.2 namespaces and encodings.
-0	Remove SOAP bindings when not explicitly set by directives.
-C	Generate client-side code only (do not generate server-side code). If you specify both -C and -S at the command line, neither client nor server code is generated.
-S	Generate server-side code (do not generate client-side code). If you specify both -C and -S at the command line, neither client nor server code is generated.

Option	Description
-T	Generate server auto-test code.
-L	Do not generate **soapClientLib**/**soapServerLib**.
-a	Use the value of the **SOAPAction** HTTP header to dispatch the method at the server side.
-A	Require **SOAPAction** to invoke server-side operations.
-b	Serialize byte arrays **char**[*N*] as a string.
-c	Generate C source code.
-d *path*	Save files in the directory specified by *path* .
-e	Generate SOAP RPC encoding style bindings.
-f *N*	A file split of N XML serializer implementations for each file (N >= 10).
-h	Display help information.
-i	Generate service proxies and objects inherited from the **soap** structure.
-I *path*	Use *path* (s) for **#import**. Multiple import directories can be listed, separated by colons (:).
-j	Generate C++ service proxies and objects that can share a **soap** struct.
-k	Generate data structure walkers (experimental).
-l	Generate linkable modules (experimental).
-m	Generate Matlab(TM) code for the MEX compiler.
-n	Use service name to rename service functions and **namespace** table.
-p *name*	Save files with the prefix *name* instead of **soap**.
-q *name*	Use *name* as the C++ namespace for **decls**, excluding custom serializers.
-Q *name*	Use *name* as the C++ **namespace** for **decls**, including custom serializers.
-s	Generate deserialization code with strict XML validation checks.

Option	Description
-t	Generate code to send typed messages (messages with the **xsi:type** attribute - SOAP/XML messaging).
-u	Uncomment comments in WSDL/schema output by suppressing XML comments.
-v	Display version information.
-w	Do not generate WSDL and schema files.
-x	Do not generate sample XML message files.
-y	Include C/C++ type access information in sample XML messages.
-z1	Generate deprecated old-style C++ service proxies and objects.

Parent topic: SOAP Application Development Tools on page 16

2.3. Message Encodings

Web Services conforms to the interoperability requirements of the Web Services Interoperability Organization (WS-I) Basic Profile 1.0.

WS-I Compliance

The WS-I Basic Profile outlines non-proprietary specifications for Web services, promoting interoperability among services. The WS-I Basic Profile disallows the use of SOAP encodings and requires instead document-literal encodings for better interoperability.

Document-Literal Versus SOAP Encodings

To conform to the WS-I Basic Profile, Web Services allows you to choose between SOAP encodings (SOAP 1.1) and document-literal encodings (SOAP 1.2). By default, document-literal encodings are generated; however, you can use the -e option if you want to develop a service that uses SOAP encodings or for backward compatibility.

For more information on the WS-I Basic Profile, see the Basic Profile 1.0 document at http://www.ws-i.org/Profiles/BasicProfile-1.0.html.

For information on gSOAP's conformance to Basic Profile 1.0, see the Compliance Report at http://www.cs.fsu.edu/~engelen/soap.html.

For information on compiler directives to specify SOAP encoding, see *gSOAP User Guide*.

Parent topic: SOAP Application Development Tools on page 16

2.4. SOAP Project Compilation Flags

Compile SOAP application source files (generated SOAP files and client and server applications) with flags.

You can use either the Wind River Compiler or the GNU compiler to compile SOAP application source files.

WNDRVR

Note: Web Services supports only the gSOAP 2.8.14 compile-time flags noted in this section. Other flags are not supported.

-DSOAP_DEBUG or -DDEBUG

This option enables message logging for code debugging purposes. This causes your code to create three log files. The files are created at the root of the VxWorks file system.

Table 1. SOAP Application Debugging Log Files

Log File	Log File Contents
SENT.log	SOAP message content that is transmitted by your application.
RECV.log	SOAP message content that is received by your application.
TEST.log	Messages concerning the operation of your application.

For details on this option, see *gSOAP User Guide*.

-DWITH_GZIP or -DWITH_ZLIB

This option enables support for **gzip** compression formats, as provided by **zlib**. For details on this option, see *gSOAP User Guide*.

-DWITH_IPV6

This option enables support for IPv6 in the SOAP run-time code in Web Services.

Note: The **-DWITH_IPV6** SOAP compile-time flag does not enable IPv6 support in the network stack. You must enable and build IPv6 support into VxWorks. For information on IPv6 support, see the *VxWorks 7 Network Stack Programmer's Guide:* IPv6 Support in a VIP.

-DWITH_OPENSSL

This option enables support for OpenSSL encryption of HTTP traffic and allows you to create secure SOAP Web services and clients. For details on this option, see *gSOAP User Guide*.

Note: The -DWITH_OPENSSL option does not enable SSL support in the kernel. For information on SSL support, see the VxWorks 7 SSL Programmer's Guide.

-DWMW_RPM_IO

This option allows for the integration of Web Services with the Web Server product in the Wind River CLI, Web, MIBway component. For more information, see Using the Wind River Web Server on page 34.

-DWITH_SOAPDEFS_H

Use this option if you specify compile definitions in the separate **soapdefs.h** file. For details on this option, see *gSOAP User Guide*.

-UWITH_LEANER and -DWITH_LEAN

Due to the large size of the **soap** structure, Web Services projects are built with the -DWITH_LEANER option enabled by default in the SOAP makefile. This removes a number of features that are not essential to basic SOAP operations.

-DWITH_LEANER disables or limits the following features:

- I/O timeouts

  Note: Many socket operations already obey some form of timeout handling, such as a connection timeout.

- HTTP keep-alive
- HTTP cookies
- HTTP authentication
- HTTP chunked output (but input is allowed)
- HTTP compressed output (but input is allowed when compiled with gzip. See -DWITH_GZIP or -DWITH_ZLIB)
- send/receive timeouts
- socket flags (**soap.socket_flag**, **soap.connect_flag**, **soap.bind_flag**, and **soap.accept_flag**)
- canonical XML output
- logging
- TCP/IP and HTTP error diagnostic messages
- support for **time_t** serialization
- support for **LONG64/ULONG64** serialization (use **typedef long xsd__long**)
- DIME and MIME attachment handling
- **wchar_t*** serialization
- support for XML DOM operations

To enable any of the features that are disabled by -DWITH_LEANER, you must specify -UWITH_LEANER. To do so, open the **Build Properties** for your project, go to the **Build Macros** tab, and enter -UWITH_LEANER for the **DEFINES** macro.

Note: If you build your SOAP application without the WITH_LEANER SOAP compiler option, you may need to increase your task stack size to 150 KB or more.

For more information on -DWITH_LEAN and -DWITH_LEANER, see *gSOAP User Guide*.

Parent topic: SOAP Application Development Tools on page 16

3. SOAP Application Development Process

Web Services facilitates the creation of SOAP server and client applications.

This section provides an overview of SOAP application development. Subsequent sections provide development tips, information on integration with other Web services and other Wind River technologies, and a list of SOAP run-time flags for use with your application.

Both SOAP server applications and client applications can be created as either kernel applications that execute in kernel-memory space in kernel mode or as RTP applications that execute in user-memory space as processes in user mode. You can integrate SOAP facilities with XML application projects.

SOAP server applications require a SOAP header file that you have created.

SOAP client applications require either a WSDL document published by the SOAP service that the client will use or a manually creater SOAP header file that provides access to the SOAP service. For more information, see SOAP Application Development Tools on page 16.

For general information about developing DKM and RTP applications, see the following documentation:

- VxWorks 7 Platform Programmer's Guide
- VxWorks 7 Application Programmer's Guide
- VxWorks 7 Kernel Application and RTP Application Projects Guide

Developing SOAP Server Applications on page 28 To develop a SOAP server application, you must create SOAP source files and, optionally, a WSDL document.
SOAP Server Code Example on page 29 Web Services provides an example of server code.
SOAP Server Files on page 30 The following C/C++ files must be included in a SOAP server application.
Developing SOAP Client Applications on page 30 To create a SOAP client application, you must create a SOAP header file and SOAP source files.
SOAP Client Code Example on page 31 Web Services provides an example of client code that calls a remote SOAP service.
SOAP Client Files on page 32 The following C/C++ files must be included in a SOAP client application.

3.1. Developing SOAP Server Applications

To develop a SOAP server application, you must create SOAP source files and, optionally, a WSDL document.

About This Task

For information on the required source files, see SOAP Server Files on page 30.

Before You Begin

You must have configured VxWorks with SOAP support. For more information, see Configuring VxWorks with SOAP Support on page 12.

Procedure

1. If you want to run your application as a DKM or RTP application, create a DKM or RTP project based on the VIP that you created in Configuring VxWorks with SOAP Support on page 12.
2. Describe the interface of your SOAP service in a SOAP header file.
3. Generate SOAP server source code using the generated SOAP header file.

   For more information, see Generating SOAP Source Files on page 19.

   If you want to provide other enterprise Web services systems with access to your application, use the wsdl2h utility to generate a WSDL document for your SOAP service from your application header file.

4. Import the **stdsoap2.c** file into your project.

   The file is located in the following directory.

   installDir **\vxworks-7\pkgs_v2\app\gsoap-***x.x.x.x* **\soap\src**

5. Develop your custom server functionality.

   For more information, see SOAP Server Code Example on page 29.

WNDRVR

Results

For an example of running a SOAP server application, see Creating and Running the SOAP Server on page 39.

Parent topic: SOAP Application Development Process on page 27

3.2. SOAP Server Code Example

Web Services provides an example of server code.

About This Task

The following description refers to the **echoStringServer.c** example, which is provided in the following directory:

installDir **/vxworks/pkgs_v2/app/gsoap/core/src/tutorials/basics**

Procedure

1. Declare variables to contain client (or *slave*) and server (or *master*) network sockets.

```
int masterSocket, slaveSocket;
```

2. Declare the SOAP object that will contain your SOAP environment.

```
struct soap soap;
```

3. Use the soap_init( ) function to initialize your SOAP object.

```
soap_init(&soap);
```

4. Bind the master socket to a network port and perform error checking.

   NULL means bind to all available IP addresses on the local machine. The port is typically the HTTP port, which is port 80.

```
masterSocket = soap_bind(&soap, NULL, port, 100);
```

5. Begin listening for client connections.

```
slaveSocket = soap_accept(&soap);
```

6. Use the soap_serve( ) function to begin serving any calls to publicly declared SOAP services.

```
soap_serve(&soap);
```

7. Use the soap_end( ) function to clean up client resources once the SOAP service is done.

```
soap_end(&soap);
```

> **Note:** Steps 5 on page 29 to 7 on page 29 are typically performed in an infinite while loop.

Parent topic: SOAP Application Development Process on page 27

WNDRVR

3.3. SOAP Server Files

The following C/C++ files must be included in a SOAP server application.

The first is your custom code, the next two you generate from a header file, and the last you import into your project.

myServer **.c(pp)**

This file contains the custom code you have developed for your SOAP server application.

soapC.c(pp)

This file contains serializers and deserializers for the specified data structures (as specified in the input header file). The serialization/deserialization is used for (de)marshalling the data between the native and user-defined C/C++ types and the XML format of the SOAP messages. The SOAP source code generator creates this file.

soapServer.c(pp)

This file contains skeleton routines. SOAP requests (from the client application) are handled by the skeleton routines, which interact with the server application to return a SOAP response to the client application. The SOAP source code generator creates this file.

stdsoap2.c

This library provides the APIs for a SOAP run-time environment. You must import this file into your project..

Note: To provide other enterprise Web services systems with access to your application, you must also publish your SOAP service's source-generated WSDL file.

Parent topic: SOAP Application Development Process on page 27

3.4. Developing SOAP Client Applications

To create a SOAP client application, you must create a SOAP header file and SOAP source files.

About This Task

For information on the required source files, see SOAP Client Files on page 32.

Before You Begin

If you are generating a SOAP header file from a published WSDL document, you must obtain the WSDL document from the SOAP service that your client will connect to. You can typically find such a file on a Web site associated with that service. For information on integrating Web Services with other enterprise Web services technologies using the WSDL importer facilities, see Integrating with Other Web Services on page 34.

You must have configured VxWorks with SOAP support. For more information, see Configuring VxWorks with SOAP Support on page 12.

Procedure

1. If you want to run your application as a DKM or RTP application, create a DKM or RTP project based on the VIP that you created in Configuring VxWorks with SOAP Support on page 12.
2. Integrate a SOAP header file into your application using one of the following methods.

WNDRVR

- If you want to use a published WSDL document from a SOAP server, see Generating SOAP Header Files from a WSDL Document on page 17.
- If you want to use a file that you have annotated with SOAP header information, see Importing Existing SOAP Header Files on page 19.

3. Using the SOAP header file, generate SOAP client source code.

   For more information, see Generating SOAP Source Files on page 19 or SOAP Command-Line Tools on page 21.

4. Import the **stdsoap2.c** file into your project.

   The file is located in the following directory.

   installDir **\vxworks-7\pkgs_v2\app\gsoap-***x.x.x.x* **\soap\src**

5. Include the **-l soap** and **-l net** libraries in your project.
6. Develop your custom client function to call and manipulate data obtained from the remote SOAP service.

   For more information, see SOAP Client Code Example on page 31.

Results

For an example of running a SOAP server application, see Running the Client RTP Application on page 42.

Parent topic: SOAP Application Development Process on page 27

3.5. SOAP Client Code Example

Web Services provides an example of client code that calls a remote SOAP service.

The following description refers to the **echoStringClient.c** example, which is provided in the following directory:

installDir **/vxworks-7/pkgs_v2/app/gsoap/core/src/tutorials/basics**

1. Declare a SOAP object where your SOAP environment is stored.

   ```
   struct soap soap;
   ```

2. Declare any variables needed to contain data acquired from the SOAP service you are interfacing with (storage for the returned string).

   ```
   char *_return;
   ```

3. Use the soap_init( ) function to initialize your SOAP object.

   ```
   soap_init(&soap);
   ```

4. Call the SOAP service.
   A typical SOAP call takes the following form:

   ```
   soap_call_namespace __SOAP_service_function_name(argument list) ;
   ```

   For example, the following is a SOAP call to the echoString service:

WNDRVR

```
soap_call_ns1__echoString(&soap, server, "", &echoString, &_return);
```

5. Perform error checking.
   For example:

```
if (soap.error)
soap_print_fault(&soap, stderr);
else
printf("echo : %s\n", _return);
```

6. Clean up client resources using the soap_destroy( ) and soap_end( ) functions.
7. Exit the function.

```
return 0;
```

Parent topic: SOAP Application Development Process on page 27

3.6. SOAP Client Files

The following C/C++ files must be included in a SOAP client application.

The first is your custom code, the next two you generate from a header file, and the last you must import into your project.

*myClient***.c(pp)**

This file contains the custom code developed for your SOAP client application.

For more information, see SOAP Client Code Example on page 31.

soapC.c(pp)

This file contains serializers and deserializers for the specified data structures (as specified in the input header file). The serialization/deserialization is used for (de-)marshalling the data between the native and user-defined C/C++ types and the XML format of the SOAP messages. The SOAP source code generator creates this file.

soapClient.c(pp)

This file contains the client stub routines and proxies (interfaces to the stub routines) for all remote methods that are callable by the client application. The invocation of the stub routines by the client application invokes the corresponding service at the remote end. The SOAP source code generator creates this file.

stdsoap2.c

This library provides the APIs for a SOAP run-time environment. You must import this file into your project.

Parent topic: SOAP Application Development Process on page 27

4. Application Development Tips

When designing SOAP applications, consider certain optimizations and issues.

Persistent HTTP Connections (HTTP Keep-Alives)

If you need your SOAP client to make multiple calls to the same HTTP server, use HTTP keep-alive socket connections on the client-side. To use HTTP keep-alive socket connections, set the **SOAP_IO_KEEPALIVE** flag for both input and output modes. You must also specify the **-UWITH_LEANER** compiler option. For information on **-UWITH_LEANER**, see SOAP Project Compilation Flags on page 25.

Data Compression in Transmission

For more efficient network data transfers, use gzip data compression to compress HTTP data. For more information, see Using the zlib Library on page 35

Memory-Handling Issues

Remove any temporary data that is no longer needed. For more information on memory management, see *gSOAP User Guide*.

39 Network Optimization

To optimize network throughput, consider the following issues:

- You can increase the buffer size by modifying the **SOAP_BUFLEN** macro in the **stdsoap2.c**.
- Enabling HTTP server keep-alives can aid in throughput from client and server HTTP code.
- You may be able to make good use of HTTP chunk transfers in your SOAP server code.

Stack Size

The following issues exist concerning stack size:

- As with almost any application, you must set aside enough memory for your program stack. Wind River recommends a minimum stack size of approximately 50KB.
- To reduce stack consumption, it is useful to allocate the SOAP data structure in a static fashion.

5. Debugging SOAP Applications

This section describes possible compilation and run-time errors.

Compilation Errors

The following is a list of possible compilation errors and their possible solutions:

"../soapH.h:8:21: stdsoap2.h: No such file or directory"

> This message indicates either that you have not used the Middleware Configurator (**File** > **New** > **VxWorks Middleware Component**) to include the SOAP component, or that your SOAP code is in a subdirectory that does not contain **soap.makefile**. See 3.6.1 SOAP Workbench Development, p.19.

"soapdefs.h: No such file or directory"

> Ensure that the compilation flag **-DWITH_SOAPDEFS_H** is not set; or, if you need to set it, ensure that the directory location of the soapdefs.h file is in your include path.

"multiple definition of 'namespaces'"

If you are building client and server code in the same image, ensure that only either the client or server code includes the namespace **.nsmap** file, but not both.

Logging

If you need to use logging, be sure to have a large file system set aside to capture the logging messages. Depending on how long the application runs and how much traffic it experiences, log files can sometimes grow to larger than 80MB. Use logging for debugging purposes only, not for operational code.

Client or Servers Hang or Generate No Output

Enable the telnet server or PC console and target shell components. Set the target shell component parameter **SHELL_STACK_SIZE** to **100000**. This combination ensures that all **stdio** is available to the user and that sufficient stack space is available to the SOAP application.

Applications Crash or Hang

Ensure that the stack size available to the SOAP application is a minimum of 50KB. Invoke the checkStack command in the shell to verify that a stack overrun has not occurred.

6. Integrating with Other Web Services

You can integrate Web Services with other enterprise Web services technologies using the wsdl2h utility.

About This Task

The following example describes a typical use case.

Procedure

1. Develop an application in the .NET environment.
2. Generate WSDL files using the .NET tools.
3. Use the WSDL importer to generate SOAP header files.
4. Use the SOAP source code generator to generate code.
5. Add your application-specific code.
6. Run your application on a VxWorks target and connect to the .NET application.

7. Using the Wind River Web Server

Web Services contains its own basic Web server by default.

To take advantage of a fuller-featured server, you can integrate SOAP with the Wind River Web Server product.

Note: If you use the Wind River Web Server with Web Services, you must run the Wind River Web Server in the same memory space (kernel or RTP) as the SOAP server.

For a set of detailed step-by-step instructions on integrating Wind River SOAP with the Wind River Web Server, see *installDir* **/vxworks-7/pkgs_v2/app/gsoap-**x.x.x.x **/core/src/tutorials/wmw/README.txt**.

For more information on the Wind River Web Server, see VxWorks 7 CLI, Web, MIBway Programmer's Guide.

8. Using the zlib Library

You can add the gzip compression feature to your system independent of the Wind River Web Server by adding the **zlib** library files to your project.

To do so, add the files from the following directory to your project.

installDir **/vxworks-7/pkgs_v2/app/webcli/core/src/tutorialstarget/src/wrn/wm/common/zlib**

Then use the **-DWITH_ZLIB** compiler flag when you build your system.

9. SOAP Run-Time Flags

You can set the following SOAP run-time flags in your application; the flags are a subset of those provided by gSOAP.

For more information—including which functions use the flags—see *gSOAP User Guide*.

SOAP_IO_FLUSH (inbound)

Disable buffering and flush output (default for all file-based output).

SOAP_IO_BUFFER (inbound)

Enable buffering (default for all socket-oriented connections).

SOAP_IO_STORE (inbound)

Store the entire message to calculate HTTP content length.

SOAP_IO_CHUNK (outbound)

Use HTTP chunk encoding.

SOAP_IO_LENGTH (outbound)

Require a priori calculation of content length (automatic).

SOAP_IO_KEEPALIVE (inbound and outbound)

Attempt to keep socket connections alive.

SOAP_IO_UDP (inbound and outbound)

Use UDP (datagram) transport. The maximum message length is **SOAP_BUFLEN**.

SOAP_ENC_XML (outbound)

Use plain XML encoding without HTTP headers (useful with **SOAP_ENC_ZLIB**).

SOAP_ENC_DIME (outbound)

Use DIME encoding (automatic when DIME attachments are used).

WNDRVR

SOAP_ENC_MIME (outbound)

Use MIME encoding (activate using **soap_set_mime**).

SOAP_ENC_MTOM (outbound)

Use MTOM XOP attachments (instead of DIME).

SOAP_ENC_ZLIB (outbound)

Compress data using **zlib** (deflate or **gzip** format encoding).

SOAP_ENC_SSL (inbound and outbound)

Encrypt encoding with SSL. This happens automatically when both endpoints of the SOAP transmission have HTTPS identifiers.

SOAP_XML_INDENT (outbound)

Produce indented XML output.

SOAP_XML_CANONICAL (outbound)

Produce canonical XML output.

SOAP_XML_DEFAULTNS (outbound)

Produces **xmlns="..."** default binding namespaced output.

SOAP_XML_IGNORENS (inbound)

Ignore the use of XML namespaces in the input.

SOAP_XML_STRICT (inbound)

Use XML strict validation (fault upon encountering unknown XML elements instead of omitting unknown elements when parsing XML).

SOAP_XML_TREE (inbound)

Ignore id attributes (do not resolve id-ref).

SOAP_XML_TREE (outbound)

Serialize data as XML trees (no multi-ref; duplicate data when necessary).

SOAP_XML_GRAPH (outbound)

Serialize data as an XML graph with inline multi-ref. This is a default setting if you compile for SOAP 1.2 compliance.

SOAP_XML_NIL (outbound)

Serialize NULL data as XML nil elements (omit by default).

SOAP_XML_NOTYPE (outbound)

Disable **xsi:type** attributes.

SOAP_C_NOIOB (inbound)

Do not fault with **SOAP_IOB** when an array is received that does not fit in a fixed size.

SOAP_C_UTFSTRING (inbound and outbound)

Serialize and deserialize 8-bit strings as is. Strings must have UTF-8 encoded content.

SOAP_C_MBSTRING (inbound and outbound)

Activate multi-byte character support.

WNDRVR

SOAP_C_NILSTRING (outbound)

Serialize empty strings as nil (ommited element).

You may use only one of the following I/O settings at a time:

- SOAP_IO_FLUSH
- SOAP_IO_BUFFER
- SOAP_IO_STORE
- SOAP_IO_CHUNK

You may use only one of these serialization flags at a time:

- SOAP_XML_TREE
- SOAP_XML_GRAPH

WNDRVR

6. XML AND SOAP TUTORIALS

1. Introduction

Web Services includes tutorials and sample code to assist in developing applications.

This chapter describes the tutorials on using XML facilities and on developing SOAP applications with XML, and provides step-by-step instructions for developing SOAP server and client applications. The source code for the tutorials is at *installDir* **/vxworks-7/pkgs_v2/app/gsoap-**x.x.x.x **/core/src/tutorials**. This directory also contains several other tutorials in addition to those described in this chapter.

2. XML Examples

Web Services provides example code from which you can learn basic supported XML functions and the APIs that are most commonly used to parse an XML document. The examples provide a starting point on using the XML parser and the XML pull parser.

The Example XML Application Development on page 14 describes a high-level procedure for developing a typical XML parser application, but additional XML examples are located in the following directory:

installDir **/vxworks-7/pkgs_v2/app/expat-**x.x.x.x **/src/tutorials/examples**

3. SOAP with XML Example

Web Services provides example code to from which you can learn how to use the XML parser in a SOAP application.

The example code is located in the following directory:

installDir **/vxworks-7/pkgs_v2/app/gsoap-**x.x.x.x **/core/src/tutorials/soapWithXML**

4. SOAP Client and Server Tutorial

This tutorial demonstrates how to create SOAP client and server applications using Workbench and Web Services. You use the VxWorks simulator as a target system.

In this tutorial, the server is implemented as a kernel application, and the client as a real-time process (RTP) application. The server runs in kernel space in kernel mode, and the client runs in user space in user mode. The two communicate across the memory boundary between kernel space and user space. The client transmits a string to the server, which the server echoes back to the client. Workbench displays the server activity in the kernel shell window, while displaying the client echo strings in the Workbench console window.

The tutorial is based on the **echoStringServer.c** and **echoStringClient.c** files, which are part of the Web Services installation. In addition to providing the basis for this tutorial, the files are designed to be used as templates for typical SOAP server and client functions. They include calls and processes generally used by client and server applications, and you can use them as the basis for your own development activity.

Additional application support code is generated from the **echoString.h** file, which provides information about application services and methods. The generated code provides the skeleton and stub routines, proxies, serializers, and deserializers that are required for

building the client and server applications with the **echoStringServer.c** and **echoStringClient.c** files. Note that Web Services also provides facilities (which are not used in this tutorial) for generating a SOAP header file from a WSDL file.

This tutorial can be used with hardware targets. With minor modifications, it can also be used in a multi-node scenario (using either VxWorks simulators or hardware targets), in which the server runs on one node and the client on another.

For information about simulating a multi-node network environment with VxWorks simulators (including a tutorial that can be used with this one), see the VxWorks 7 VxSim User's Guide.

Note that for hardware targets, you must configure VxWorks with the components required for real-time processes (as the default image does not provide this support) and configure the DNS resolver. For information on configuring VxWorks for real-time processes, see the *VxWorks 7 Platform Programmer's Guide: Configuring VxWorks for Real-time Processes*. For information on the DNS resolver, see the VxWorks 7 Network Stack Programmer's Guide.

5. Creating and Running the SOAP Server

In this part of the tutorial, the SOAP server is created as a downloadable kernel module (DKM) project, which allows you to use the default VxWorks simulator without having to rebuild the operating system with the application code.

In general, DKM projects are useful for rapid development of kernel applications.

1. Create a VSB project for the server.
2. Create a VIP based on the VSB project.
3. Create a DKM project based on the VSB project and add server application code.
4. Create a connection to the target and run the server application.

Creating a VSB Project on page 39 You must include the required VSB layers for the SOAP server application.
Creating a VIP on page 40 You must include the required VIP components for the SOAP server application.
Creating a DKM Project on page 40 You must import the example files into a DKM project and generate additional source files for the SOAP server application.
Running the Server DKM Application on page 41 You must run the server application with VxSim.

5.1. Creating a VSB Project

You must include the required VSB layers for the SOAP server application.

Procedure

1. Start Workbench and create a VxWorks source build project with the following configuration:
   - Name the project **soapServerVSB**.
   - Base the project on a CPU.
   - Select the SIMNT CPU type.

   Note: For a hardware target, you can select the appropriate target and toolchain when creating the project.

2. In the Project Explorer, open the **Source Build Configuration** of the **soapServerVSB** project.
3. Enable the **GSOAP** layer.
4. Save the new configuration.
5. Build the VSB.

WNDRVR

Parent topic: Creating and Running the SOAP Server on page 39

5.2. Creating a VIP

You must include the required VIP components for the SOAP server application.

Procedure

1. Create a VIP with the following configuration:
   - Name the project **soapServerVIP**.
   - Base the project on the **soapServerVSB** VSB project.
2. In the Project Explorer, open the **Source Build Configuration** of the **soapServerVIP** project.
3. Add the **INCLUDE_GSOAP_DEMO** component to the VIP.
4. Add the **INCLUDE_ANSI_TIME** component to the VIP.
5. Save the new configuration.
6. Build the VIP.

Parent topic: Creating and Running the SOAP Server on page 39

5.3. Creating a DKM Project

You must import the example files into a DKM project and generate additional source files for the SOAP server application.

Procedure

1. Create a DKM project with the following configurations:
   - Name the project **soapServerDKM**.
   - Base the project on the **soapServerVSB** VSB project.
2. Import the **echoString.h** and **echoStringServer.c** sample files from the following directory into the **soapServerDKM** DKM project.

   installDir **/vxworks-7/pkgs_v2/app/gsoap-***x.x.x.x* **/core/src/tutorials/basics**

3. In the Project Explorer view, right-click the **echoString.h** file and select **Generate SOAP Source Code**.
4. For the Add SOAP source files to project selection, clear the **Client** check box, and then click **OK**.

   The Soapcpp2 Plug-in dialog box appears. The soapcpp2 utility displays the file names and file paths that it generates from the definitions in the **echoString.h** file.

5. Click **OK**.
6. In the Project Explorer view, open the **echoStringServer.c** file.
7. Modify the file as follows.

```
#if !defined(_WRS_KERNEL) || 1
```

8. Save the file.
9. Build the DKM project.

Parent topic: Creating and Running the SOAP Server on page 39

WNDRVR

5.4. Running the Server DKM Application

You must run the server application with VxSim.

Procedure

1. Create a new VxWorks Simulator connection with the following configuration:
   - Name the connection **VxSim SOAP**.
   - Select the **soapServerVIP/default/vxWorks** kernel image.
   - Select the **Connect on Finish** check box.
2. In the Project Explorer view, right-click the **soapServerDKM** DKM project and select **Run/Debug Kernel Task**.
3. Run the DKM with the following configuration:
   - Select the **echoStringServer** entry point.
   - Enter an argument of **80**.
     This argument specifies that the server run on port 80.
4. Switch to the kernel shell.

   The kernel shell displays the following message.

```
-> Socket connection successful: master socket = 8
```

5. In the kernel shell, execute the i command to verify that the **tSOAPsrvr** server task is running.

```
-> i
NAME ENTRY TID PRI STATUS PC SP ERRNO DELAY
---------- ------------ -------- --- ---------- -------- -------- ------- -----
tJobTask 10093610 103b7310 0 PEND 10129d03 105bff48 0 0
tExcTask 100929d0 101a6140 0 PEND 10129d03 101a6040 0 0
tLogTask logTask 103bb288 0 PEND 101282f6 105ffee8 0 0
tNbioLog 10094440 103bf808 0 PEND 10129d03 1063fef8 0 0
tShell0 shellTask 105548c8 1 READY 10131410 1084e0a8 0 0
tWdbTask wdbTask 1047d328 3 PEND 10129d03 107ffee8 0 0
tAioIoTask> aioIoTask 103da978 50 PEND 1012a703 106bfd18 0 0
tAioIoTask> aioIoTask 103ddda8 50 PEND 1012a703 106ffd18 0 0
tNet0 ipcomNetTask 103df490 50 PEND 1013ede8 1073ff94 0 0
ipcom_sysl> 10057210 103fd310 50 PEND 1012a703 107bfe48 0 0
tAioWait aioWaitTask 103d7478 51 PEND 10129d03 1067feb8 0 0
tSOAPsrvr echoStringSe 116ff808 150 PEND 1005032b 108ee7c8 0 0
value = 0 = 0x0
```

Note: Leave the VxWorks simulator running.

Parent topic: Creating and Running the SOAP Server on page 39

6. Creating and Running the SOAP Client

In this part of the tutorial, the SOAP client application is created as a VxWorks real-time process (RTP) application.

The basic steps for creating and starting the SOAP client are as follows:

1. Create an RTP project for the client and add client application code.
2. Run the client application.

Creating an RTP Project on page 42 You must import the example files into an RTP project and generate additional source files for the SOAP client application.
Running the Client RTP Application on page 42 You must run the client application with VxSim.

6.1. Creating an RTP Project

You must import the example files into an RTP project and generate additional source files for the SOAP client application.

Procedure

1. Create an RTP project with the following configuration:
   - Name the project **soapClientRTP**.
   - Base the project on the **soapServerVSB** VSB project.
2. In the **soapClientRTP** RTP project, open the **rtp.c** file and rename the main( ) function.
3. Import the **echoString.h** and **echoStringClient.c** sample files from the following directory into the **soapClientRTP** RTP project.

   installDir **/vxworks-7/pkgs_v2/app/gsoap-***x.x.x.x* **/core/src/tutorials/basics**

4. In the Project Explorer view, right-click the **echoString.h** file and select **Generate SOAP Source Code**.
5. For the Add SOAP source files to project selection, clear the **Server** check box, and then click **OK**.

   The Soapcpp2 Plug-in dialog box appears. The soapcpp2 utility displays the file names and file paths that it generates from the definitions in the **echoString.h** file.

6. Click **OK**.
7. Import the **stdsoap2.c** file into your project.

   The file is located in the following directory.

   installDir **\vxworks-7\pkgs_v2\app\gsoap-***x.x.x.x* **\soap\src**

8. Add the **-l soap** and **-l net** libraries to all build specs.
9. Build the RTP project.

Parent topic: Creating and Running the SOAP Client on page 41

6.2. Running the Client RTP Application

You must run the client application with VxSim.

Procedure

1. In the Project Explorer view, right-click the **soapClientRTP** RTP project and select **Run/Debug Kernel Task**.
2. Run the RTP with the following configuration:
   - In the **Arguments** field, type the string or strings that you want to send to be echoed by the server
     For example, **Hello, world**.
   - In the advanced options, change the initial task stack size to **0x150000**.

WNDRVR

> **Note:** This SOAP application does not include the WITH_LEANER SOAP compiler option, so you must increase the task stack size to **150000** or more. For more information, see SOAP Project Compilation Flags on page 25.

After a moment, the kernel shell displays status information about the connection, and the server echoes the strings you specified.

For example:

```
-> Socket connection successful: slave socket = 13
Socket connection successful: slave socket = 13
```

Workbench displays the output from the client in the console window. Each command-line argument that you specified is echoed as a separate string.

For example:

```
echo : Hello,
echo : world
```

Parent topic: Creating and Running the SOAP Client on page 41